

*

1. NMAP Analysis (TCP SYN Scan -sS):

→ Using `nmap -sS 192.168.1.1/24`, we scanned the local network. Open ports found included 22 (SSH), 80 (HTTP), and 443 (HTTPS).

- Port 22 runs SSH service for secure remote login.
- Port 80 hosts web servers for unencrypted traffic.
- Port 443 serves HTTPS with encrypted communication.

The SYN scan is stealthy and doesn't complete the full TCP handshake.

2. Comparative Scanning C-sT vs -AJ:

-> The command `nmap -sT 192.168.1.5` gives basic port and service info, while `nmap -A 192.168.1.5` reveals OS details, versions, scripts, and traceroute.

The aggressive scan C-AJ provides deeper insights like service banners, running OS, and possible vulnerabilities.

It is more likely to be detected by firewalls or IDS.

3. NMAP Commands Table :

Command	Purpose	Output Summary
<code>nmap -sP 192.168.1.0/24</code>	Ping Scan to find live hosts	Lists active IPs on the network
<code>nmap -sV 192.168.1.5</code>	Detect service versions	Shows version details of services
<code>nmap -O 192.168.1.5</code>	OS detection	Guesses OS type and version
<code>nmap -Pn 192.168.1.5</code>	Scan with no ping	Scans even if host doesn't respond to ping
<code>nmap --script vuln 192.168.1.5</code>	Runs vulnerability detection scripts	Lists known vulnerabilities

4. Wireshark Filter Creation:

-> Captured traffic using Wireshark and applied filter: `http`.

It displayed only HTTP packets (port 80)

From filtered packets, we saw GET/POST requests, hostnames, user-agent, and requested URLs

This data reveals what websites were accessed and basic client info.

Useful for analyzing unencrypted web activity.

5. IP Tracking with Wireshark:

-> During a file download, Wireshark showed communication between the source IP (web server) and destination IP.

Protocol used was HTTP (unsecured), observed on Port 80. If HTTPS was used, the protocol would be TLS.

6. DVWA Setup and Report:

-> Steps:

- 1 -> Download DVWA
- 2 -> Place in htdocs/
- 3 -> Configure config.inc.php
- 4 -> Start Apache & MySQL
- 5 -> Access via localhost/dvwa and set DB

7. Reflected XSS Attack:

-> Payload used: `<script>alert('XSS')</script>` in input field.

On submission, an alert box pops up - showing that user input was reflected in the response without sanitization.

Prevention: Use input validation and HTML encoding (e.g., `htmlspecialchars()` in PHP).

8. SQL Injection Experiment :

→ Payload: ` OR 1=1 -- bypassed login by turning the SQL query into a condition that always returns true.

Original query: `SELECT * FROM users
WHERE username = '' OR 1=1 --`

This causes the DB to return all users.

Prevention: Use prepared statements or parameterized queries to safely handle input.

9. Security Level Testing in DVWA:

Security Level	SQL Injection Result	XSS Result
Low	Easily bypassed login	XSS script runs without filter
Medium	Input partially sanitized	Basic filtering, can bypass
High	Strong input validation	Javascript stripped low blocked

Higher levels use techniques like regex validation, output encoding, and secure queries to defend against attacks.

10. Ethical Analysis:

- > Practicing attacks in DVWA ensures a safe, legal environment without harming real systems. In real-world penetration testing, written permission is mandatory to avoid legal violations. Ethically, testing without consent is hacking and can lead to prosecution. Controlled environments help students learn responsible cybersecurity skills and understand attack impacts.